

Exercise 1: Implementing the Singleton Pattern

```
J Logger.java 1 X J LoggerTest.java 1
SingletonPatternExample > J Logger.java > Logger
1 public class Logger {
2
3     private static Logger instance;
4
5     private Logger(){}
6
7     public static Logger getInstance(){
8         if(instance==null){
9             instance=new Logger();
10        }
11        return instance;
12    }
13
14    public void log(String message){
15        System.out.println("LOG: "+message);
16    }
17 }
```

```
SingletonPatternExample > J LoggerTest.java > LoggerTest
1 public class LoggerTest {
2
3     public static void main(String[] args) {
4         Logger singletonPatternExample.Logger.getInstance()
5         Logger logger1=Logger.getInstance();
6         Logger logger2=Logger.getInstance();
7
8         logger1.log(message: "Application started");
9         logger2.log(message: "User logged in");
10
11         if(logger1==logger2){
12             System.out.println(x: "Both references point to the same Logger instance.");
13         }else{
14             System.out.println(x: "Different Logger instances exist.");
15         }
16     }
17 }
```

RESULT :

```

PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\WEEK_1_MANDATORY\SingletonPatternExample> javac *.java
PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\WEEK_1_MANDATORY\SingletonPatternExample> java LoggerTest
LOG: Application started
LOG: User logged in
Both references point to the same Logger instance.
❖ PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\WEEK_1_MANDATORY\SingletonPatternExample>

```

Exercise 2: Implementing the Factory Method Pattern

```
FactoryMethodPatternExample > Document.java > Document
1 public interface Document {
2     String getType();
3 }
```

Document.java 1 WordDocument.java 1 X PdfDocument.java 1

FactoryMethodPatternExample > WordDocument.java > WordDocument

```
1 public class WordDocument implements Document {  
2  
3     public String getType(){  
4         return "Word Document";  
5     }  
6
```

```
FactoryMethodPatternExample > PdfDocument.java > PdfDocument
1 public class PdfDocument implements Document {
2
3     public String getType(){
4         return "PDF Document";
5     }
6 }
```

```
java 1 WordDocument.java 1 PdfDocument.java 1 ExcelDocument.java 1 X
FactoryMethodPatternExample > ExcelDocument.java > ExcelDocument
1 public class ExcelDocument implements Document {
2
3     public String getType(){
4         return "Excel Document";
5     }
6 }
```

```
FactoryMethodPatternExample > J ExcelFactory.java > ExcelFactory
1 public class ExcelFactory extends DocumentFactory {
2
3     public Document createDocument(){
4         return new ExcelDocument();
5     }
6 }
```

```
FactoryMethodPatternExample > J DocumentFactory.java > DocumentFactory
1 public abstract class DocumentFactory {
2     public abstract Document createDocument();
3 }
```

```
Document.java 1 J DocumentFactory.java 1 J WordFactory.java 1 X J PdfFactory.java 1
FactoryMethodPatternExample > J WordFactory.java > WordFactory
1 public class WordFactory extends DocumentFactory {
2
3     public Document createDocument(){
4         return new WordDocument();
5     }
6 }
```

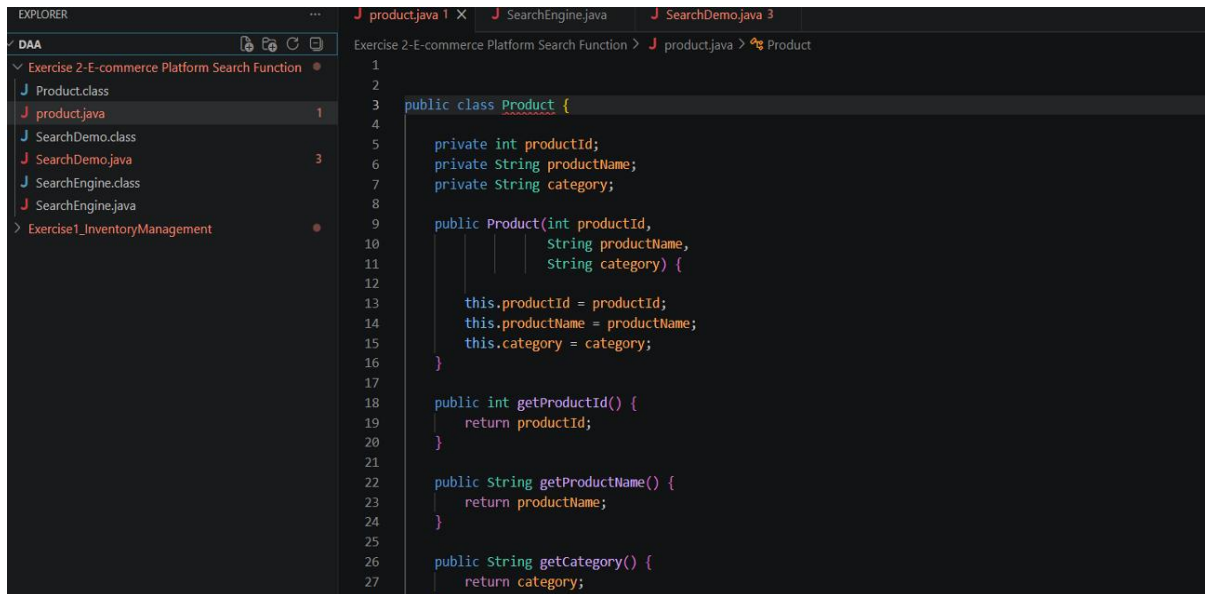
```
Document.java 1 J DocumentFactory.java 1 J WordFactory.java 1 X J PdfFactory.java 1
FactoryMethodPatternExample > J PdfFactory.java > PdfFactory
1 public class PdfFactory extends DocumentFactory {
2
3     public Document createDocument(){
4         return new PdfDocument();
5     }
6 }
```

```
1 J WordFactory.java 1 J PdfFactory.java 1 J ExcelFactory.java 1 J FactoryDemo.java 1 X
FactoryMethodPatternExample > J FactoryDemo.java > FactoryDemo
1 public class FactoryDemo {
2
3     Run | Debug
4     public static void main(String[] args){
5
6         DocumentFactory wordFactory=new WordFactory();
7         DocumentFactory pdfFactory=new PdfFactory();
8         DocumentFactory excelFactory=new ExcelFactory();
9
10        Document d1=wordFactory.createDocument();
11        Document d2=pdfFactory.createDocument();
12        Document d3=excelFactory.createDocument();
13
14        System.out.println("Created: "+d1.getType());
15        System.out.println("Created: "+d2.getType());
16        System.out.println("Created: "+d3.getType());
17    }
18 }
```

RESULT :

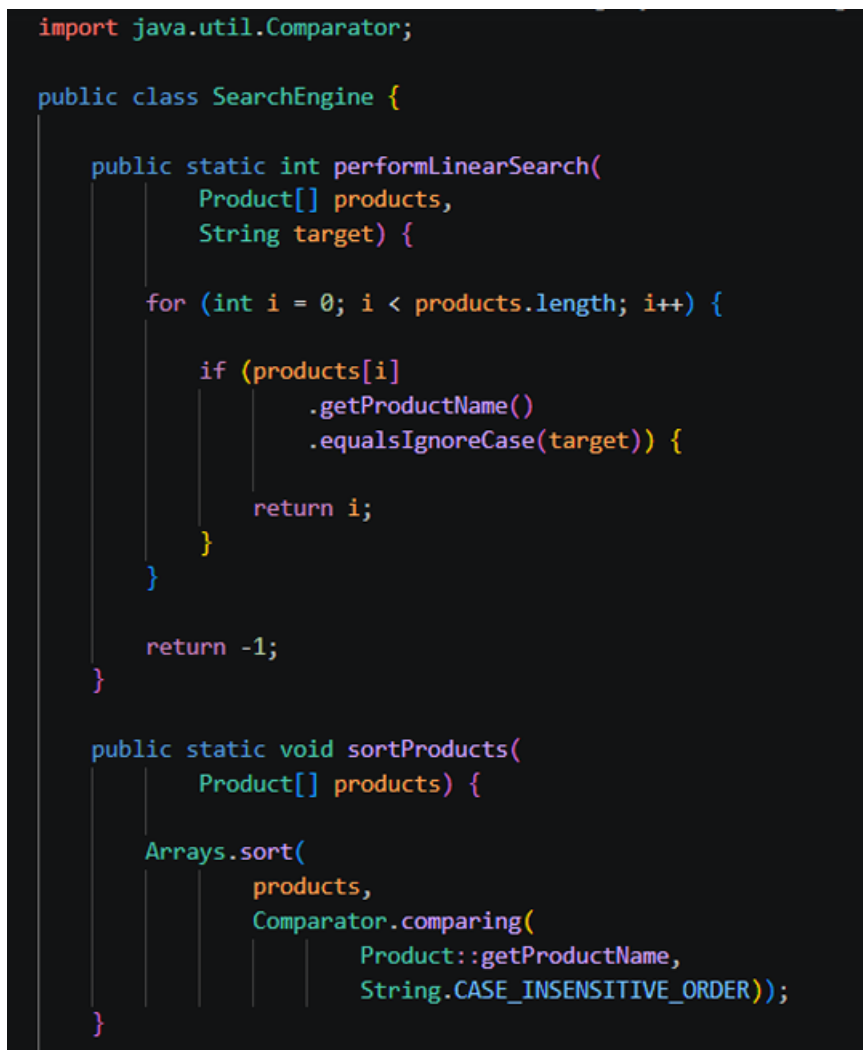
```
PROBLEMS 11 OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell - FactoryMethodPatternExample + - [ ] [ ] [ ]
• PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\WEEK_1_MANDATORY\FactoryMethodPatternExample> javac *.java
• PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\WEEK_1_MANDATORY\FactoryMethodPatternExample> java FactoryDemo
Created: Word Document
Created: PDF Document
Created: Excel Document
❖ PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\WEEK_1_MANDATORY\FactoryMethodPatternExample> [ ]
```

Exercise 2: E-commerce Platform Search Function



The screenshot shows an IDE with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'DAA' with a sub-project 'Exercise 2-E-commerce Platform Search Function'. Inside this sub-project, there are four files: 'Product.class', 'product.java', 'SearchDemo.class', and 'SearchEngine.class'. The 'product.java' file is selected, and its content is displayed in the code editor. The code defines a 'Product' class with three private attributes: 'productId' (int), 'productName' (String), and 'category' (String). It includes a constructor that initializes these attributes and three getter methods: 'getProductId()', 'getProductName()', and 'getCategory()'.

```
1
2
3 public class Product {
4
5     private int productId;
6     private String productName;
7     private String category;
8
9     public Product(int productId,
10                    String productName,
11                    String category) {
12
13         this.productId = productId;
14         this.productName = productName;
15         this.category = category;
16     }
17
18     public int getProductId() {
19         return productId;
20     }
21
22     public String getProductName() {
23         return productName;
24     }
25
26     public String getCategory() {
27         return category;
28     }
29 }
```



The screenshot shows a code editor with the implementation of the 'SearchEngine' class. It starts with an import statement for 'java.util.Comparator'. The class contains two static methods. The first method, 'performLinearSearch', takes an array of 'Product' objects and a 'String target' as input. It iterates through the array, checking if the 'productName' of each product matches the target (case-insensitive). If a match is found, it returns the index 'i'. If no match is found, it returns -1. The second method, 'sortProducts', takes an array of 'Product' objects and uses 'Arrays.sort' with a 'Comparator.comparing' function to sort the array based on the 'productName' in case-insensitive order.

```
import java.util.Comparator;

public class SearchEngine {

    public static int performLinearSearch(
        Product[] products,
        String target) {

        for (int i = 0; i < products.length; i++) {

            if (products[i]
                .getProductName()
                .equalsIgnoreCase(target)) {

                return i;
            }
        }

        return -1;
    }

    public static void sortProducts(
        Product[] products) {

        Arrays.sort(
            products,
            Comparator.comparing(
                Product::getProductName,
                String.CASE_INSENSITIVE_ORDER));
    }
}
```

```

public static int performBinarySearch(
    Product[] products,
    String target) {

    int left = 0;
    int right = products.length - 1;

    while (left <= right) {

        int mid =
            left + (right - left) / 2;

        int result =
            products[mid]
                .getProductName()
                .compareToIgnoreCase(target);

        if (result == 0) {
            return mid;
        }

        if (result < 0) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }

    return -1;
}

```

The screenshot shows an IDE with a project named "Exercise 2-E-commerce Platform Search Function". The project contains several files: Product.class, product.java, SearchDemo.class, SearchDemo.java, SearchEngine.class, SearchEngine.java, and Exercise1_InventoryManagement. The file "SearchDemo.java" is selected and open in the editor. The code in the editor is as follows:

```

3 public class SearchDemo {
4
5     Run | Debug
6     public static void main(String[] args) {
7         Product[] products = new Product[] {
8             new Product(productId: 1, productName: "Laptop", category: "Electronics"),
9             new Product(productId: 2, productName: "Mouse", category: "Electronics"),
10            new Product(productId: 3, productName: "Keyboard", category: "Electronics"),
11            new Product(productId: 4, productName: "Smartphone", category: "Electronics"),
12            new Product(productId: 5, productName: "Headphones", category: "Electronics")
13        };
14
15        String target = "mouse";
16
17        System.out.println(x: "Linear Search (unsorted):");
18        int idx1 = SearchEngine.performLinearSearch(products, target);
19        System.out.println("Target: " + target + " -> index = " + idx1);
20
21        // Binary search requires sorted array
22        System.out.println(x: "\nSorting products by name...");
23        SearchEngine.sortProducts(products);
24
25        System.out.println(x: "Binary Search (sorted):");
26        int idx2 = SearchEngine.performBinarySearch(products, target);
27        System.out.println("Target: " + target + " -> index = " + idx2);
28
29        if (idx2 != -1) {
30            System.out.println("Found product: " + products[idx2].getProductName()
31                + " (category=" + products[idx2].getCategory() + ")");
32        }
33    }
34 }

```

RESULT :

```
PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\DAA\Exercise 2-E-commerce Platform Search Function> cd C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\DAA\Exercise 2-E-commerce Platform Search Function; javac Product.java SearchEngine.java SearchDemo.java
PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\DAA\Exercise 2-E-commerce Platform Search Function>
PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\DAA\Exercise 2-E-commerce Platform Search Function> java SearchDemo
Linear Search (unsorted):
Target: mouse -> index = 1

Sorting products by name...
Binary Search (sorted):
Target: mouse -> index = 3
Found product: Mouse (category=Electronics)
PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\DAA\Exercise 2-E-commerce Platform Search Function> 
```

Exercise 7: Financial Forecasting

```
ForecastCalculator.java 1 ForecastDemo.java 1 X
Exercise7_FinancialForecasting > J ForecastDemo.java > ForecastDemo
1 public class ForecastDemo {
2
3     Run | Debug
4     public static void main(String[] args){
5
6         double currentAmount=50000;
7         double growthRate=0.08;
8         int years=5;
9
10        double futureValue=
11            ForecastCalculator.predictValue(
12                currentAmount,
13                growthRate,
14                years
15            );
16
17        System.out.println("Current Amount : "+currentAmount);
18        System.out.println("Growth Rate : "+(growthRate*100)+"%");
19        System.out.println("Years : "+years);
20        System.out.println("Predicted Future Value : "+futureValue);
21    }
```

```
ForecastCalculator.java 1 X ForecastDemo.java 1
Exercise7_FinancialForecasting > J ForecastCalculator.java > ForecastCalculator
1 public class ForecastCalculator {
2
3     public static double predictValue(double amount,double growthRate,int years){
4         if(years==0)
5             return amount;
6
7         return predictValue(amount*(1+growthRate),growthRate,years-1);
8     }
9 }
```

RESULT:

```
PROBLEMS 28 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\DAA\Exercise7_FinancialForecasting> javac *.java
PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\DAA\Exercise7_FinancialForecasting> java ForecastDemo
Current Amount : 50000.0
Growth Rate : 8.0%
Years : 5
Predicted Future Value : 73466.40384000003
PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\DAA\Exercise7_FinancialForecasting> 
```

Exercise 1: Control Structures

Scenario 1: Apply Discount to Senior Citizens

```
BEGIN
  FOR rec IN (
    SELECT l.LoanID,c.DOB
    FROM Customers c
    JOIN Loans l
    ON c.CustomerID=l.CustomerID
  )
  LOOP
    IF TRUNC(MONTHS_BETWEEN(SYSDATE,rec.DOB)/12) > 60 THEN
      UPDATE Loans
      SET InterestRate=InterestRate-1
      WHERE LoanID=rec.LoanID;

      DBMS_OUTPUT.PUT_LINE('Discount applied to Loan ID '||rec.LoanID);
    ELSE
      DBMS_OUTPUT.PUT_LINE('No discount for Loan ID '||rec.LoanID);
    END IF;
  END LOOP;

  COMMIT;
END;
```

RESULT :

```
SQL> BEGIN
      FOR rec IN (
        SELECT l.LoanID,c.DOB
        FROM Customers c...
Show more...

Discount applied to Loan ID 1
No discount for Loan ID 2

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.017
```

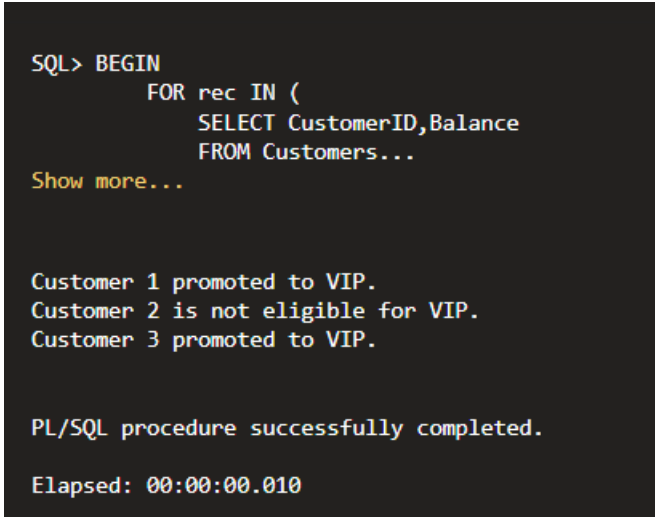
Scenario 2: Promote Customers to VIP

```
BEGIN
  FOR rec IN (
    SELECT CustomerID,Balance
    FROM Customers
  )
  LOOP
    IF rec.Balance > 10000 THEN
      UPDATE Customers
      SET IsVIP='Y'
      WHERE CustomerID=rec.CustomerID;

      DBMS_OUTPUT.PUT_LINE(
        'Customer '||rec.CustomerID||' promoted to VIP'
      );
    ELSE
      DBMS_OUTPUT.PUT_LINE(
        'Customer '||rec.CustomerID||' is not eligible for VIP.'
      );
    END IF;
  END LOOP;

  COMMIT;
END;
```

RESULT :



```
SQL> BEGIN
      FOR rec IN (
        SELECT CustomerID,Balance
        FROM Customers...
      )
      LOOP
        IF rec.Balance > 10000 THEN
          UPDATE Customers
          SET IsVIP='Y'
          WHERE CustomerID=rec.CustomerID;

          DBMS_OUTPUT.PUT_LINE(
            'Customer '||rec.CustomerID||' promoted to VIP'
          );
        ELSE
          DBMS_OUTPUT.PUT_LINE(
            'Customer '||rec.CustomerID||' is not eligible for VIP.'
          );
        END IF;
      END LOOP;

      COMMIT;
    END;

Customer 1 promoted to VIP.
Customer 2 is not eligible for VIP.
Customer 3 promoted to VIP.

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.010
```

Scenario 3: Loan Due Reminder

```
BEGIN
  FOR rec IN (
    SELECT l.LoanID,
           c.Name,
           l.EndDate
    FROM Loans l
    JOIN Customers c
    ON l.CustomerID=c.CustomerID
    WHERE l.EndDate BETWEEN SYSDATE AND SYSDATE+30
  )
  LOOP
```

```

DBMS_OUTPUT.PUT_LINE(
  'Reminder: Loan ID '||rec.LoanID||
  ' for '||rec.Name||
  ' is due on '||
  TO_CHAR(rec.EndDate,'YYYY-MM-DD')
);
END LOOP;
END;

```

RESULT :

```

SQL> BEGIN
      FOR rec IN (
        SELECT l.LoanID,
              c.Name,...
Show more...

Reminder: Loan ID 1 for Nimith B is due on 2026-07-14

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.019

```

Exercise 3: Stored Procedures

Procedure: ProcessMonthlyInterest

```

CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest IS
BEGIN
  FOR acc IN (
    SELECT AccountID, Balance
    FROM Accounts
    WHERE AccountType='Savings'
  ) LOOP

    UPDATE Accounts
    SET Balance = Balance + (acc.Balance * 0.01),
        LastModified = SYSDATE
    WHERE AccountID = acc.AccountID;

    DBMS_OUTPUT.PUT_LINE(
      'Interest applied to Account ID ' || acc.AccountID
    );

  END LOOP;

  COMMIT;
END;

```

Procedure: UpdateEmployeeBonus

```

CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(
  p_department IN VARCHAR2,
  p_bonus_percent IN NUMBER
)
IS

```

```

BEGIN
  FOR emp IN (
    SELECT EmployeeID, Salary
    FROM Employees
    WHERE Department = p_department
  ) LOOP

    UPDATE Employees
    SET Salary = Salary + (emp.Salary * p_bonus_percent / 100)
    WHERE EmployeeID = emp.EmployeeID;

    DBMS_OUTPUT.PUT_LINE(
      'Bonus applied to Employee ID ' || emp.EmployeeID
    );

  END LOOP;

  COMMIT;
END;

```

Procedure: TransferFunds :

```

CREATE OR REPLACE PROCEDURE TransferFunds(
  p_from_account IN NUMBER,
  p_to_account IN NUMBER,
  p_amount IN NUMBER
)
IS
  v_balance NUMBER;
BEGIN

  SELECT Balance
  INTO v_balance
  FROM Accounts
  WHERE AccountID = p_from_account;

  IF v_balance < p_amount THEN
    RAISE_APPLICATION_ERROR(-20001,'Insufficient funds');
  END IF;

  UPDATE Accounts
  SET Balance = Balance - p_amount,
  LastModified = SYSDATE
  WHERE AccountID = p_from_account;

  UPDATE Accounts
  SET Balance = Balance + p_amount,
  LastModified = SYSDATE
  WHERE AccountID = p_to_account;

  COMMIT;

  DBMS_OUTPUT.PUT_LINE(
    'Transferred ' || p_amount ||
    ' from Account ' || p_from_account ||
    ' to Account ' || p_to_account
  );

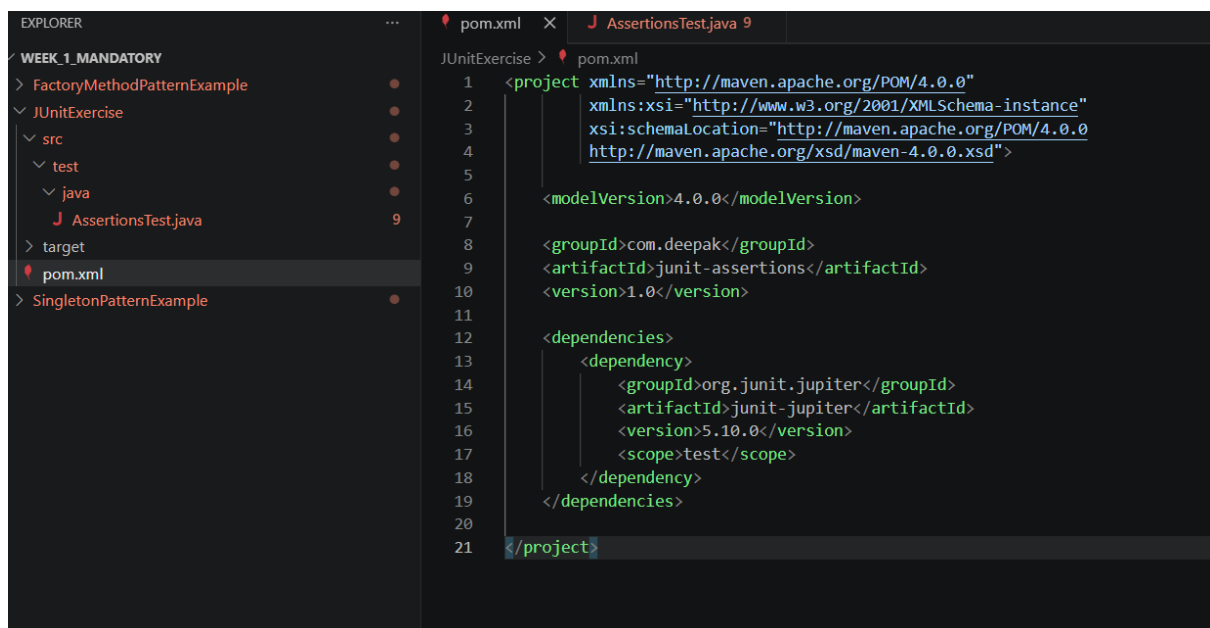
  EXCEPTION
  WHEN OTHERS THEN
    ROLLBACK;
  DBMS_OUTPUT.PUT_LINE(SQLERRM);
END;

```

RESULT :

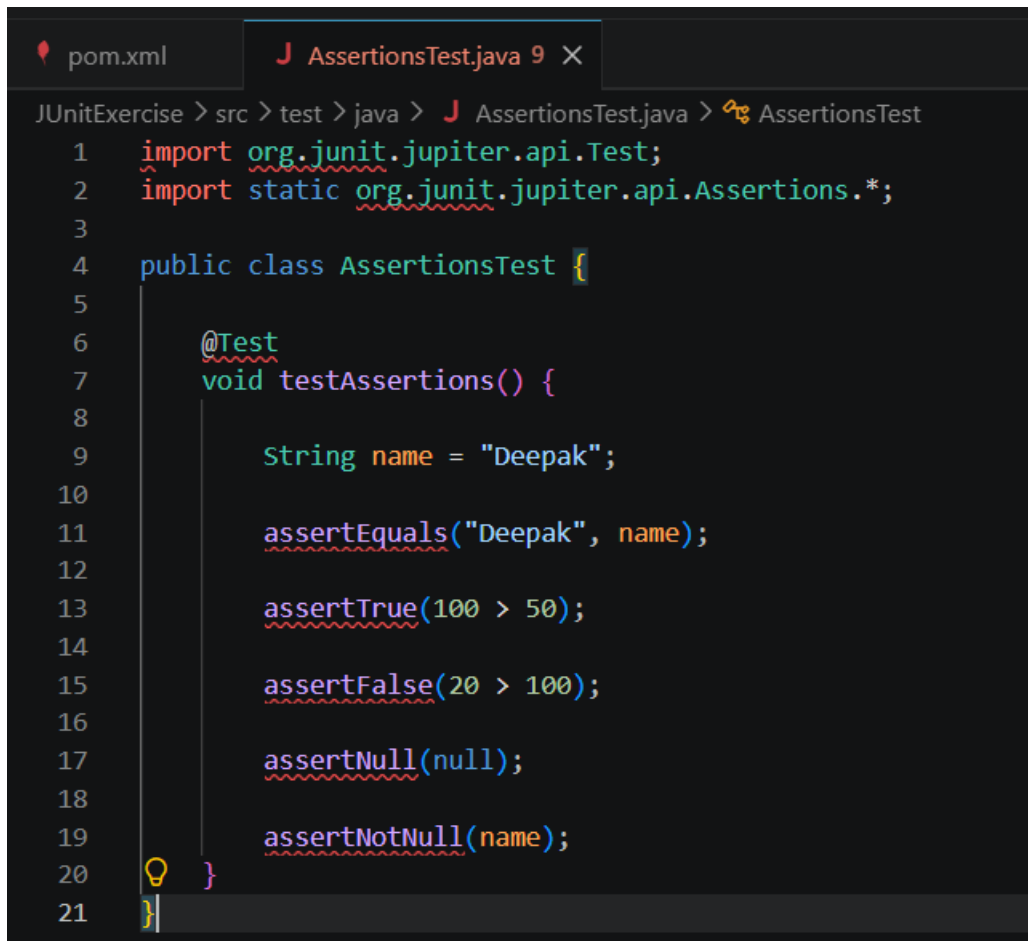
```
--- Calling ProcessMonthlyInterest ---  
Interest applied to Account ID 101  
Interest applied to Account ID 102  
  
--- Calling UpdateEmployeeBonus for HR, 10% ---  
Bonus applied to Employee ID 1  
  
--- Calling TransferFunds: 200 from 101 to 102 ---  
Transferred 200 from Account 101 to Account 102  
  
PL/SQL procedure successfully completed.  
  
Elapsed: 00:00:00.052
```

Exercise 3: Assertions in Junit



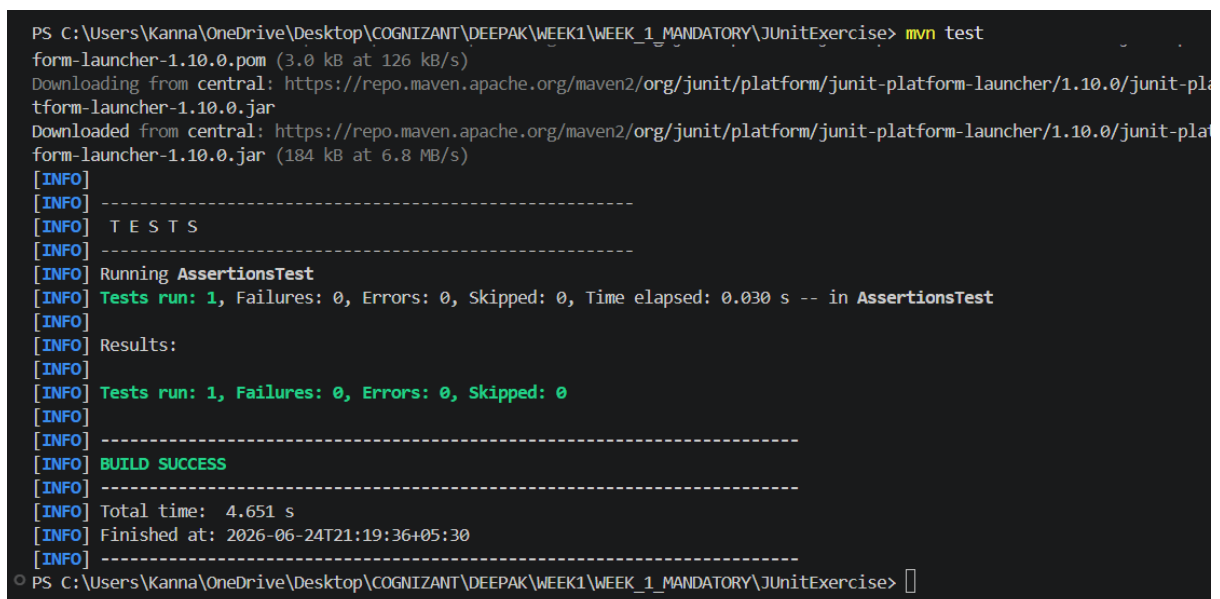
The screenshot shows an IDE with a project explorer on the left and a code editor on the right. The project explorer shows a project named 'WEEK_1_MANDATORY' with sub-projects 'FactoryMethodPatternExample', 'JUnitExercise', and 'SingletonPatternExample'. The 'JUnitExercise' project is expanded, showing 'src', 'test', and 'java' directories. The 'pom.xml' file is selected in the project explorer. The code editor displays the contents of 'pom.xml' for 'JUnitExercise', which is a Maven project configuration for 'junit-assertions'.

```
1 <project xmlns="http://maven.apache.org/POM/4.0.0"  
2     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
3     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0  
4       http://maven.apache.org/xsd/maven-4.0.0.xsd">  
5  
6     <modelVersion>4.0.0</modelVersion>  
7  
8     <groupId>com.deepak</groupId>  
9     <artifactId>junit-assertions</artifactId>  
10    <version>1.0</version>  
11  
12    <dependencies>  
13        <dependency>  
14            <groupId>org.junit.jupiter</groupId>  
15            <artifactId>junit-jupiter</artifactId>  
16            <version>5.10.0</version>  
17            <scope>test</scope>  
18        </dependency>  
19    </dependencies>  
20  
21 </project>
```



```
pom.xml  AssertionsTest.java 9 X
JUnitExercise > src > test > java > AssertionsTest.java > AssertionsTest
1  import org.junit.jupiter.api.Test;
2  import static org.junit.jupiter.api.Assertions.*;
3
4  public class AssertionsTest {
5
6      @Test
7      void testAssertions() {
8
9          String name = "Deepak";
10
11          assertEquals("Deepak", name);
12
13          assertTrue(100 > 50);
14
15          assertFalse(20 > 100);
16
17          assertNull(null);
18
19          assertNotNull(name);
20      }
21 }
```

RESULT :



```
PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\WEEK_1_MANDATORY\JUnitExercise> mvn test
[INFO] Downloading from central: https://repo.maven.apache.org/maven2/org/junit/platform/junit-platform-launcher/1.10.0/junit-pla
[INFO] form-launcher-1.10.0.jar
[INFO] Downloaded from central: https://repo.maven.apache.org/maven2/org/junit/platform/junit-platform-launcher/1.10.0/junit-pla
[INFO] form-launcher-1.10.0.jar (184 kB at 6.8 MB/s)
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running AssertionsTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.030 s -- in AssertionsTest
[INFO] Results:
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 4.651 s
[INFO] Finished at: 2026-06-24T21:19:36+05:30
[INFO] -----
PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\WEEK_1_MANDATORY\JUnitExercise> 
```

Exercise 4: Arrange-Act-Assert (AAA) Pattern, Test Fixtures, Setup and Teardown Methods in Junit

```
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

public class StudentTest {

    private Student student;

    @BeforeEach
    void setUp(){
        student = new Student();
        System.out.println("Setup Complete");
    }

    @AfterEach
    void tearDown(){
        System.out.println("Test Finished");
    }

    @Test
    void testPassGrade(){

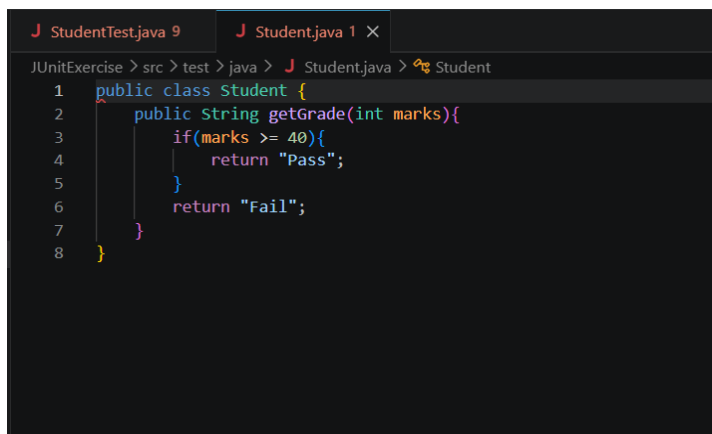
        int marks = 75;
        String result = student.getGrade(marks);

        assertEquals("Pass", result);
    }

    @Test
    void testFailGrade(){
        int marks = 20;

        String result = student.getGrade(marks);

        assertEquals("Fail", result);
    }
}
```

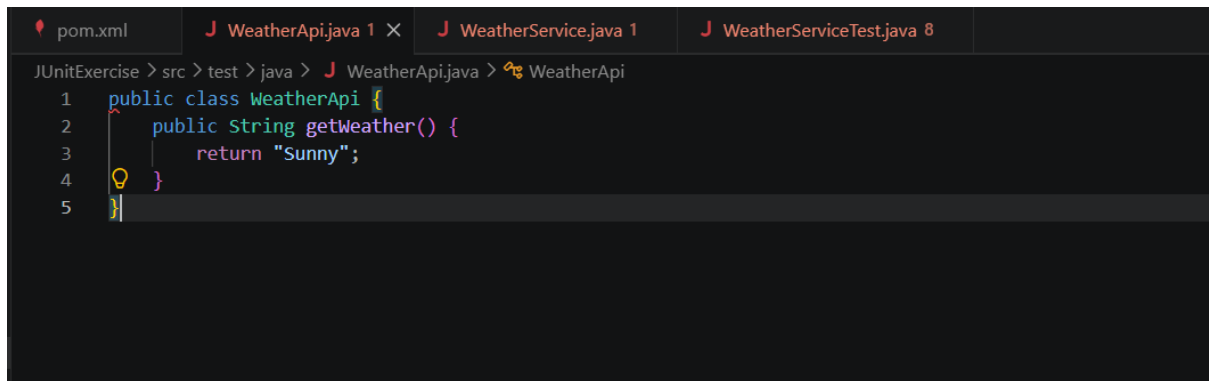


```
JUnitExercise > src > test > java > J Student.java > Student
1 public class Student {
2     public String getGrade(int marks){
3         if(marks >= 40){
4             return "Pass";
5         }
6         return "Fail";
7     }
8 }
```

RESULT :

```
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running AssertionsTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.030 s -- in AssertionsTest
[INFO] Running StudentTest
Setup Complete
Test Finished
Setup Complete
Test Finished
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.011 s -- in StudentTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 1.406 s
[INFO] Finished at: 2026-06-24T21:24:40+05:30
[INFO] -----
PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\WEEK_1_MANDATORY\JUnitExercise>
```

Exercise 1: Mocking and Stubbing



```
pom.xml  J WeatherApi.java 1 X  J WeatherService.java 1  J WeatherServiceTest.java 8
JUnitExercise > src > test > java > J WeatherApi.java > WeatherApi
1 public class WeatherApi {
2     public String getWeather() {
3         return "Sunny";
4     }
5 }
```

```
pom.xml WeatherApi.java 1 WeatherService.java 1 X WeatherServiceTest.java 8
JUnitExercise > src > test > java > WeatherService.java > WeatherService
1 public class WeatherService {
2
3     private WeatherApi api;
4
5     public WeatherService(WeatherApi api){
6         this.api = api;
7     }
8
9     public String fetchWeather(){
10         return api.getWeather();
11     }
12 }
```

```
JUnitExercise > src > test > java > WeatherServiceTest.java > WeatherServiceTest
1 import org.junit.jupiter.api.Test;
2 import org.mockito.Mockito;
3
4 import static org.junit.jupiter.api.Assertions.assertEquals;
5
6 public class WeatherServiceTest {
7
8     @Test
9     void testWeatherApi(){
10
11         WeatherApi mockApi = Mockito.mock(WeatherApi.class);
12
13         Mockito.when(mockApi.getWeather())
14             .thenReturn("Rainy");
15
16         WeatherService service =
17             new WeatherService(mockApi);
18
19         String result = service.fetchWeather();
20
21         assertEquals("Rainy", result);
22     }
23 }
```

RESULT :

```
[INFO] Running WeatherServiceTest
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build what is described in Mockito's documentation: https://javadoc.io/doc/org.mockito/mockito-core/latest/org.mockito/Mockito.html#0.3
WARNING: A Java agent has been loaded dynamically (c:\Users\Kanna\.m2\repository\net\bytebuddy\byte-buddy-agent\1.15.4\byte-buddy-agent-1.15.4.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.724 s -- in WeatherServiceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 3.527 s
[INFO] Finished at: 2026-06-24T21:33:40+05:30
[INFO]
```

Exercise 2: Verifying Interactions

pom.xml J NotificationService.java 1 X J UserManager.java 1 J UserManagerTest.java 6

JUnitExercise > src > test > java > J NotificationService.java > NotificationService

```
1 public class NotificationService {
2     public void sendMessage(String message) {
3         System.out.println(message);
4     }
5 }
```

JUnitExercise > src > test > java > J UserManager.java > UserManager

```
1 public class UserManager {
2     private NotificationService notificationService;
3
4     public UserManager(NotificationService notificationService) {
5         this.notificationService = notificationService;
6     }
7
8     public void registerUser() {
9         notificationService.sendMessage(message: "Welcome Deepak");
10    }
11 }
```

```
pom.xml  J NotificationService.java 1  J UserManager.java 1  J UserManagerTest.java 6 X
JUnitExercise > src > test > java > J UserManagerTest.java > UserManagerTest
1  import org.junit.jupiter.api.Test;
2  import static org.mockito.Mockito.*;
3
4  public class UserManagerTest {
5
6      @Test
7      void testVerifyInteraction() {
8
9          NotificationService mockService =
10             | mock(NotificationService.class);
11
12             UserManager manager =
13             | new UserManager(mockService);
14
15             manager.registerUser();
16
17             verify(mockService)
18             | .sendMessage("Welcome Deepak");
19         }
20     }
```

RESULT :

```
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.006 s -- in StudentTest
[INFO] Running UserManagerTest
```

Exercise 1: Logging Error Messages and Warning Levels

```
JUnitExercise > src > main > java > J LoggingExample.java > LoggingExample > main(String[])
1
2  import org.slf4j.Logger;
3  import org.slf4j.LoggerFactory;
4
5
6  public class LoggingExample {
7
8      private static final Logger logger =
9      | LoggerFactory.getLogger(LoggingExample.class);
10
11      Run | Debug
12      public static void main(String[] args) {
13
14          | logger.error("Database connection failed.");
15          | logger.warn("Disk space is below 20%.");
16      }
17  }
```

RESULT :

```
[INFO]
[INFO] --- exec:3.6.3:java (default-cli) @ junit-assertions ---
21:51:05.897 [LoggingExample.main()] ERROR LoggingExample -- Database connection failed.
21:51:05.899 [LoggingExample.main()] WARN LoggingExample -- Disk space is below 20%.
[INFO] -----
[INFO] BUILD SUCCESS
[INFO]
```